ensiie

Introduction à la Standard Template Library

C++ avancé

Rappels C++

- Passages d'arguments
- Patrons de classes
- Sémantique de déplacement / transfert parfait
- Lambda expressions
- Opérateurs

Rappels C & spécificités de C++

- Modules (.h/.c) & Classes (.h[pp]*/.cpp)
  - Compilation séparée des classes comme des modules**
- Passages d'arguments (1/3)
  - par valeur :

```
int f(int val)
{
    val++;
    return val;
}
```

main

```
int i = 15;
int r = f(i);
// i == 15
// r == 16
```

  - une copie de la « valeur » val est réalisée dans pile d'appel
  - cette copie est incrémentée, puis copiée dans la valeur de retour
    - La valeur de la variable passée en argument n'est jamais modifiée

* : ⚠️ .hpp est souvent réservé aux headers contenant directement une implémentation (pas de .cpp).

** : au sens général du terme et non pas au sens C++ 20.

Passages d'arguments (2/3)

- par adresse :

```
int f(int * ptr)
{
    (*ptr)++;
    return *ptr;
}
```

main

```
int i = 15;
int r = f(&i);
// i == 16
// r == 16
```

  - une copie de la « valeur » ***ptr*** est réalisée dans pile d'appel ⇒ une copie de l'adresse ***ptr***
  - Cette adresse est déréférencée (****ptr***) puis la valeur ainsi obtenue est incrémentée (**++**) *
  - la valeur de l'adresse déréférencée est copiée dans la valeur de retour
    - la valeur pointée par l'adresse passée en argument a été modifiée

*: notez les parenthèses et consultez les priorités des opérateurs en C/C++ page 17-18
(*ptr)++ ≠ *ptr++

Passages d'arguments (3/3)

- par référence :

```
int f(int & ref)
{
    ref++;
    return ref;
}
```

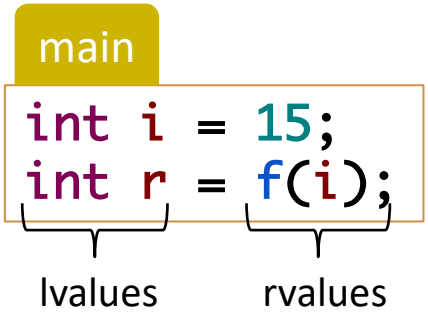
```
main
int i = 15;
int r = f(i);
// i == 16
// r == 16
```

  - une copie de l'**adresse** référencée par ***ref*** est réalisée dans pile d'appel
  - la valeur référencée par ***ref*** est incrémentée
  - la valeur référencée par ***ref*** copiée dans la valeur de retour
    - la valeur référencée par la référence passée en argument a été modifiée

- Rq : il ne peut y avoir de référence vers void ni de référence de référence.

Lvalues et Rvalues (1/2)

- Première approche :
  - lvalue : à gauche (ou à droite) d'une affectation
  - rvalue : à droite d'une affectation



- Deuxième approche :
  - **lvalue** : « locator value » toute valeur possédant un emplacement mémoire
    - On peut toujours obtenir l'adresse d'une lvalue : opérateur & : &lvalue
  - **rvalue** : Tout ce qui n'est pas une lvalue
    - Valeur constante
    - Résultat d'une expression ou d'une fonction renvoyant une valeur (mais pas une référence)
    - On ne peut pas obtenir l'adresse d'une rvalue

Lvalues et Rvalues (2/2)

- Exemples

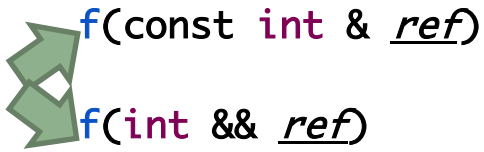
```
int & retRef() {return …;}
int retVal(){return …;}
int main(int argc, char** argv)
{
      // lvalues:
      int i = 42;
      i = 43; // ok, i is an lvalue
      int* p = &i; // ok, i is an lvalue and &i is an rvalue
      retRef() = 42; // ok, retRef() is an lvalue
      int* p1 = &retRef(); // ok, retRef() is an lvalue

    // rvalues:
    int j = 0;
    j = retVal(); // ok, retVal() is an rvalue
    int* p2 = &retVal(); // error, cannot take the address of an rvalue
    j = 42; // ok, 42 is an rvalue
    return 0;
}
```

Lvalues & rvalues references

C++11

- lvalue reference : référence à un emplacement mémoire défini par une lvalue :
  - `Type a = 3;` // Ok: a is an lvalue
  - `Type & lvra = a;` // Ok: lvalue reference to lvalue
  - ~~`Type & lvrb = 15;`~~ // Error: non const lvalue reference to an rvalue
  - `const Type & clvrc = 15;` // Ok: const lvalue reference to an rvalue
  - `Type * pa = &a;` // Ok: address of an lvalue
  - `Type * plvra = &lvra;` // Ok: address of an lvalue reference

- rvalue reference : référence (temporaire) à une rvalue :
  - ~~`Type && prvra = a;`~~ // Error: rvalue reference to an lvalue
  - `Type && rvrb = 15;` // Ok : rvalue reference to rvalue
  - `Type * prvr = &rvrb;` // Ok: address of an rvalue reference (ahem!)
  - Utilisé dans la sémantique de déplacement
  - Pour résoudre `int f(int & ref)` → `f(42)` impossible



Généricité - *Patrons de classe*

- Mot clé : **template**
- Patrons de **fonctions** et de **classes**

1. Patrons de fonctions :

   Les types génériques doivent apparaître au minimum dans les types des arguments

```
template <class C1, class C2> TypeRetour nomFonction (C1 arg1, C2 arg2, ...)
{ ... }
```

2. Patrons de classes :

```
template <class C1, class C2> class C
{
   C1 m1 ;
   void f (C2) ;
   ...
} ; ...
template <class C1, class C2> void C::f(C2 arg ) { … }
```

utilisation des mots clés `class` ou `typename` pour définir les paramètres de type

3. Instanciation du patron avec des types connus :

```
C <int, double> monInstaneDeC;
C <Compte, float> autreInstaneDeC;
```

Deux types différents

Attention, pas de conversion de type implicite, sauf entre classes héritières et classes ancêtres

→ Le lien dynamique d'héritage est préservé

Généricité - *Patrons Variadiques*

C++11

- Définition : template de classe ou de fonction dont tout ou partie de la liste des paramètres de types est variable
- Patron de Fonction / Méthode variadique :

```
template <typename T, typename ...R>
void Tableau<T>::addElement(const T & first, const R &... others)
{
    addElement(first); // Appel de la version à 1 argument
    addElement(others...); // Appel récursif
}
```

- Patron de classe variadique: `template <class ..._Tp> class tuple`
  - `tuple<int, string, Compte>`
  - `tuple<bool, int>`
- Introduction de 2 nouveaux opérateurs
  - opérateur **…** (souvent appelé « ellipse »)
    - à gauche d'un nom d'argument : désigne un « paquet de paramètres » (parameter pack) : permet de grouper [0..n) arguments
    - à droite d'un nom d'argument : sépare les paramètres (pack expansion)
  - Nouvel opérateur `sizeof…(<parameter pack name>)`
    - permet d'obtenir la taille d'un paquet de paramètres

Membres par défaut

C++11

- Membres par défaut (générés automatiquement si absents*)
  - Constructeur par défaut : `MaClasse()`
  - Constructeur de copie : `MaClasse(const MaClasse & mc)`
  - Constructeur de déplacement : `MaClasse(MaClasse && mc)`
  - Opérateur de copie : `MaClasse & operator =(const MaClasse & mc)`
  - Opérateur de déplacement : `MaClasse & operator =(MaClasse && mc)`
  - Destructeur : `virtual ~MaClasse()`
- *: Règles (🚫 = not generated, ❌ = deleted) ⇔ à déclarer explicitement

	Default Ctor	Default Copy Ctor	Default Move Ctor	Default Copy Operator	Default Move Operator	Default Dtor
Nothing	✅	✅	✅	✅	✅	✅
Any Ctor	🚫	✅	✅	✅	✅	✅
Default Ctor	🚫	✅	✅	✅	✅	✅
Copy Ctor	🚫	🚫	🚫	✅	🚫	✅
Move Ctor	🚫	❌	🚫	❌	🚫	✅
Copy Operator	✅	✅	🚫	🚫	🚫	✅
Move Operator	✅	❌	🚫	❌	🚫	✅
[Virtual] Dtor	✅	✅	🚫	✅	🚫	🚫

Sémantique de déplacement

C++03

Problème

```
MyObject o0;

MyObject o1(input);

MyObject o2(o1);
MyObject o3 = o1;

MyObject o4 = MyObject(input);

MyObject o5;

o5 = MyObject(input);
```

Constructeur par défaut

Constructeur valué

Constructeur de copie

Constructeur valué
Constructeur de copie
Destructeur

Rvalues

Constructeur valué
Opérateur de copie
Destructeur

- Comment éviter les créations / copie / destructions inutiles (surtout si celles-ci sont coûteuses en espace mémoire et/ou en temps d'exécution)
  - Idée : « déplacer » les rvalues dans les lvalues, ⇔ les lvalues subtilisent les ressources contenues dans les rvalues lorsque cela est possible grâce à des opérations spécifiques :
    - Constructeur de déplacement
    - Opérateur de déplacement

Sémantique de déplacement

C++11

- Solutions
  - Elision de copie par le compilateur (copy elision)*

    * : dépend du compilateur

    - Lorsqu'un objet temporaire est censé être (copié | déplacé) vers un objet du même type (la copie | le déplacement) est omise : l'objet temporaire est alors directement créé dans l'espace mémoire censé le recueillir.

```
MyObject o4 = MyObject(input);
 ⋮
o5 = MyObject(input);
```

Constructeur valué

Constructeur valué
Constructeur de déplacement
Destructeur

Constructeur valué
Opérateur de déplacement
Destructeur

  - Utilisation du constructeur de déplacement et de l'opérateur de déplacement (s'ils existent) **
    - Le déplacement est alors considéré comme une copie destructive (vol des ressources des rvalues, contrairement à la copie ordinaire qui laisse les rvalues dans leur état initial).

** : défini dans le langage

Transfert parfait (Perfect Forwarding)

C++11

- Pb : Transmettre un argument d'une fonction/méthode à une autre tel quel (les lvalues en tant que lvalues et les rvalues en tant que rvalues).

```
template <typename T> void foo(T & lvalue)
{
   cout << "lv = " << lvalue  << " ";
}
```

```
template <typename T> void foo(T && rvalue)
{
   cout << "rv = " << rvalue  << " ";
}
```

```
noForward(T t)
badForward1(T & t)
badForward2(const T & t)
template <typename T> void goodForward(T && t)
{
   foo(t);

   foo(std::forward<T>(t)); // sauf dans badForward2(const T &)
   //  <==> static_cast<decltype(t)&&>(t)
}
```

Pas de forwarding (copie de la valeur t passée en argument)

Fonctionne avec les lvalues mais pas avec les rvalues

Spécialisation pour les constantes (rvalues) : Mais forward<T> interdit car cast d'un **const T &** en **T &&** looses const qualifier !

On ne parle plus de « rvalue references » mais de « universal references »

Comportement attendu du **forward** :
- rvalue transmise en tant que rvalue
- lvalue transmise en tant que lvalue

```
int main(void)
{
   int i = 2;
   noForward(1);    // lv = 1  rv = 1
   noForward(i);    // lv = 2  rv = 2
   badForward1(i);  // lv = 2  rv = 2
   badForward2(1);  // lv = 1
   goodForward(1);  // lv = 1  rv = 1
   goodForward(i);  // lv = 2  lv = 2
}
```

Lambda Expressions

C++11

- Syntaxe
  - [*capture*] (*arguments*) -> *return_type* {*corps*}
  - *capture* : spécification des éléments de l'environnement capturés dans la lambda expression (et donc utilisables dans le corps).
    - [] : Rien n'est capturé : La Lambda Expression est alors considérée comme une fonction
    - [&] : Capture de toutes les variables de l'environnement par référence
    - [=] : Capture des toutes les variables de l'environnement par copie
    - [=, &foo] : Capture des variables par copie, sauf foo par référence
    - [bar] : Capture de la variable bar par copie, rien d'autre n'est capturé.
    - [this] : Capture du pointeur this de la classe englobante.
  - arguments : Pas de parenthèses si aucun argument.
  - -> *return_type* : Optionnel s'il peut être déduit du corps par le compilateur.
  - {*corps*} : corps de l'expression lambda.
- Type
  - Quel est le type d'une expression lambda ? (Si l'on veut la référencer par une variable)
    - Avec capture : `template <class Ret, class... Args> class function<Ret(Args...)>;` (voir templates variadiques page 10),
    - Sans capture : `function<Ret(Args…)>` ou n'importe quel pointeur de fonction équivalent : `Ret(*)(Args…)`.

Lambda Expressions : exemples

C++11

```
string message("Hello World");
// omission des parenthèses quand pas d'arguments et capture par ref
auto f0 = [&message]{cout << message << endl;};      // auto f0→ function<void(void)> ou void (*f1)()
f0(); // Appel de la fonction
// expression complète
auto f1 = [](int x, int y) -> int{return x + y;};    // auto f1→ function<int(int, int)> ou int (*f1)(int, int)
// type de retour implicite, inféré de par la clause return
auto f2 = [](float x, float y){return x + y;};       // auto f2→ function<float(float, float)> ou float (*f2)(float, float)
// pas de clause return, type de retour = void
auto f3 = [](int & x){++x;};                         // auto f3→ function<void(int &)> ou void (*f3)(int &)
// accès à une variable globale (bèèèh)
auto f4 = [](){++global_x;};                         // auto f4→ function<void(void)> ou void (*f4)()
```

```
vector<float> operate(const vector<float> & v1, const vector<float> & v2,
                      const function<float(float, float)> & func)
{
    …func(…, …)…
}
```

```
vector<float> v1 = {1.1f, 2.1f, 3.1f, 4.1f, 5.1f};
vector<float> v2 = {5.2f, 4.2f, 3.2f, 2.2f, 1.2f, 0.2f};
vector<float> v3 = operate(v1, v2, atan2f);
vector<float> v4 = operate(v1, v2, f2);
vector<float> v5 = operate(v1, v2, [](float x, float y) -> float {return x + y;});
```

Fonction ordinaire

Lambda référencée par f2

Lambda anonyme

Opérateurs C++ (1/2)

Priorité	Opérateur	Description	Associativité	usage	Surcharge membre / *non membre (global)*	Note
18	::	Opérateur de résolution de portée	gauche à droite	NameSpace::Classe::membre		
17	()	Parenthèses	gauche à droite	instance(…)	T & MaClasse::operator() (const unsigned int,…)	foncteurs
	[]	Tableau (subscript)		instance[…]	T & MaClasse::operator[](unsigned i)	
	.	Sélection d'un membre par un identificateur (structures et objets)		instance.membre		
	->	Sélection d'un membre par un pointeur (structures et objets)		pointeur->membre	T* MaClasse::operator->(void)	itérateurs / smart pointers
	++ --	Incrémentation post-fixée (1)		instance++	void MaClasse::operator++(int) ← dummy int	itérateurs
16	++ --	Incrémentation pré-fixée	droite à gauche	++instance	void MaClasse::operator++(void)	itérateurs
	+ -	Opérateurs + et - unaires		-instance	MaClasse & MaClasse::operator-(void) *friend MaClasse operator-(const MaClasse &);*	
	! ~	Opérateurs unaires négation logique et bit à bit		!instance	MaClasse & MaClasse::operator!(void) const; *friend MaClasse operator!(const MaClasse &);*	
	*	Déréférencement		*instance	T& MaClasse::operator* (void)	itérateurs / smart pointers
	&	Adresse		&instance	T* MaClasse::operator&(void); *friend MaClasse* operator&(MaClasse &);*	itérateurs / smart pointers
	sizeof	Opérateur de taille (d'objet ou de type)		sizeof(Type ou instance)		
	new new[]	Allocation mémoire et mémoire tableau		Type * ptr = new(user args) Type(…) Type[] tab = new[nbElements]	void *MaClasse::operator new(size_t s, user args); void* MaClasse::operator new[ ] (size_t);	allocators. implicitement déclarés en tant que méthodes de classe
	delete delete[]	Libération mémoire et mémoire tableau		Type * ptr; … ; delete ptr; Type[] ptr; … ; delete[] ptr;	void MaClasse::operator delete (void *); void MaClasse::operator delete[ ] (void*);	
15	(type)	Cast	droite à gauche	(Type)instanceDeMaClasse	MaClasse::operator Type(void)	plusieurs
14	.*	Déréfencement d'un pointeur de membre d'un objet	gauche à droite	instance.*membre		
	->*	Déréfencement d'un pointeur de membre d'un objet pointé		(pointeur->*membre)()		pointeur de fonction membre

*ptr++

Opérateurs C++ (2/2)

Priorité	Opérateur	Description	Associativité	usage	Surcharge membre / *non membre (global)*	Note
13	* / %	Multiplication, Division et Modulo	gauche à droite	instance1 * instance 2	MaClasse & MaClasse::operator*(const MaClasse &) const	
12	+ -	Addition, Soustraction	gauche à droite	instance1 + instance 2	MaClasse & MaClasse::operator+(const MaClasse &) const	
11	<< >>	Décalages de bits à gauche ou à droite	gauche à droite	instance << 1 cout << Instance cin >> instance	void MaClasse::operator<<(unsigned) *friend ostream & operator << <> (ostream &, const MaClasse &);* *friend istream & operator >> <> (istream &, MaClasse &);*	iostream
10	< <=	Opérateurs relationnels "strictement inférieur" et "inférieur ou égal"	gauche à droite	instance1 < instance2	bool MaClasse::operator<(const MaClasse &) const	
	> >=	Opérateurs relationnels "strictement supérieur" et "supérieur ou égal"		instance1 > instance2	bool MaClasse::operator>(const MaClasse &) const	
9	== !=	Opérateurs relationnels "égal" et "différent de"	gauche à droite	instance1 == instance2	bool MaClasse::operator==(const MaClasse &) const	
8	&	ET bit à bit	gauche à droite	instance1 & instance2	MaClasse & MaClasse::operator&(const MaClasse &) const	
7	^	OU exclusif bit à bit	gauche à droite	instance1 ^ instance2	"	
6	|	OU bit à bit	gauche à droite	instance1 | instance2	"	
5	&&	ET logique	gauche à droite	instance1 && instance2	bool MaClasse::operator&&(const MaClasse &) const	
4	||	OU logique	gauche à droite	instance1 || instance2	"	
3	c?t:f	Opérateur de condition ternaire	droite à gauche	(expression?instrTrue:instFalse)		
2	=	Affectation	droite à gauche	instance1 = instance2	MaClasse & operator = (const MaClasse & m)	
	+= -=	Affectation avec somme ou soustraction		instance += valeur instance1 += instance2	void operator+=(const Type &) MaClasse & operator+=(const MaClasse &)	
	*= /= %=	Affectation avec multiplication, division ou modulo		instance *= valeur ou instance	"	
	<<= >>=	Affectation avec décalages de bits		instance <<= valeur ou instance	"	
	&= ^= |=	Affectation avec ET, OU exclusif ou OU inclusif bit à bit		instance &= valeur ou instance	"	
1	,	Séquence d'expressions	gauche à droite	e=a, b, c, d ⇔ (((e=a),b),c),d	MaClasse & operator,(const Maclasse &)	concat

Opérateurs

- subscript operators ()|[] come in pairs
  - Exemple

```
template <typename T>
class Matrix
{
  protected:
    ⋮
  public:
    Matrix(const size_t i, const size_t j) : … {…}
    T operator ()(const size_t i, const size_t j) const {…}
    T & operator ()(const size_t i, const size_t j) {…}
    ⋮
}
```

const

non const

  - Utilisation

```
{
  Matrix<double> m(3,4);
  double value = m(2,3); // T operator ()(const size_t i, const size_t j) const
  m(2,3) = 45.0; // T & operator ()(const size_t i, const size_t j)
}
```

const

non const